

Avances del Proyecto

Avances del Proyecto — Sistema de IA Anticipatoria para el STC Metro

Alumno: Alan Bellon García **Asesor:** M. en Fil. C. Enrique Francisco Soto Astorga **Fecha de este reporte:** 2026-09-27 (actualizado — ver [Sección 4](#) , [Sección 5](#) , [Sección 6](#) y [Sección 7](#))

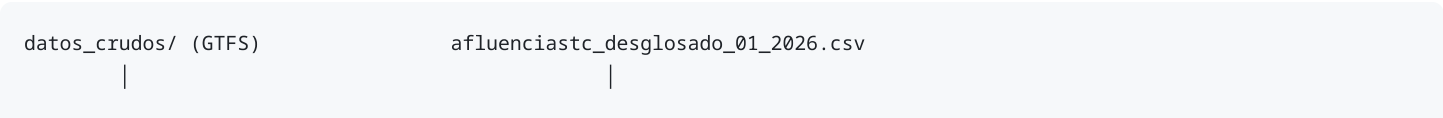
Este documento resume el estado técnico y metodológico del prototipo descrito en el anexo de titulación ("*Modelado y prototipado de un sistema de Inteligencia Artificial Anticipatoria para la estimación de estados de congestión en el STC Metro de la Ciudad de México*"), con base en el código y los datos actualmente presentes en `00_Programas/tesis_metro_ai/`.

1. Mapeo contra los objetivos de la tesis

Objetivo secundario (Anexo)	Estado	Evidencia en código
Generar dataset sintético de afluencia/disrupciones	✅ Completo (14 días sintéticos, laboral+fin de semana, eventos estocásticos con tasas calibradas, capacidad por línea)	<code>generador_sintetico_horario.py</code> , <code>simulador_congestion.py</code> , <code>datos_procesados/*.csv</code>
Representar la red como grafo con pesos	✅ Completo	<code>grafo_metro.py</code> → <code>grafo_base_metro.gexf</code>
Modelo de estimación a horizonte corto (10–60 min)	⚠️ Parcial (horizonte discreto de 1 hora, no continuo 10-60 min); comparación de 6 configuraciones; modelo fijo RandomForest + línea/tramo porque la selección por semilla es inestable (Secciones 6.4 y 7.1)	<code>entrenador_anticipatorio.py</code> → <code>modelos/modelo_anticipatorio.pkl</code>
Algoritmo de ruteo que integre la métrica predictiva	✅ Prueba de concepto funcional	<code>ruteo_anticipatorio.py</code>
Integración estimación + ruteo en prototipo funcional	⚠️ Evaluado con 30 semillas (258 horas con evento, 525,446 casos O-D): la IA pierde 5.3× menos que el reactivo (IC excluye 0) y captura 47.7% [23.2; 67.6] del ahorro posible, pero su ventaja en ahorro total no es significativa (Sección 7.3)	<code>ruteo_anticipatorio.py</code>
Explicabilidad de las recomendaciones	✅ Iniciado (importancia por permutación, agnóstica al modelo)	<code>entrenador_anticipatorio.py</code> → <code>importancia_variables.png</code>
Sistema reactivo de comparación (índice 5.5)	✅ Implementado y comparado: estático vs. reactivo vs. anticipatorio vs. oráculo	<code>ruteo_anticipatorio.py</code>

En términos del índice tentativo, el proyecto ha cubierto el **capítulo 4** completo (modelado y generación de datos) y tiene un **primer corte funcional del capítulo 5** (diseño del prototipo: arquitectura, modelo predictivo, ruteo, baseline reactivo). Falta evaluación formal de KPIs (5.4), profundizar explicabilidad (6.5) y ejecutar comparativas sistemáticas (6.4) sobre múltiples escenarios, no solo el caso de prueba único.

2. Pipeline técnico actual (orden de ejecución)





2.1 grafo_metro.py — Modelado topológico

- Filtra GTFS masivo del Valle de México por `agency_id == "METRO"` para aislar rutas, viajes y `stop_times`.
- Construye grafo no dirigido $G(V, E, W)$ con **195 nodos** (andenes/estaciones).
- Peso estático de cada arista = promedio de `departure_sec - arrival_sec` entre paradas consecutivas del mismo `trip_id`, filtrando deltas ilógicos (<0 o $>1800s$).
- Agrega aristas de **transbordo peatonal** (5 min fijos) entre andenes que comparten `stop_name`, habilitando cambios de línea en el grafo.
- Exporta a `grafo_base_metro.gexf` (formato estándar, reutilizable por NetworkX).

Limitación notable: el penalti de transbordo es una constante (5 min) igual para todas las estaciones de correspondencia, sin distinguir estaciones con transbordos largos conocidos (p. ej. Ciudad Azteca, La Raza, Tacubaya).

2.2 generador_sintetico_horario.py — Motor de demanda

- Limpia encoding (latin1 → utf-8) de estación/línea provenientes del portal de datos abiertos.
- Clasifica estaciones en 3 perfiles espaciotemporales (`origen`, `destino`, `mixto`) mediante listas fijas (hardcoded) de 10 estaciones por extremo.
- Desagrega afluencia diaria a horaria multiplicando por curvas bimodales (`peso_origen`, `peso_destino`, `peso_mixto`) calibradas manualmente para 24 horas.
- **Matriz Origen-Destino sintética** vía modelo gravitacional simplificado: calcula `atractividad_destino` según hora y perfil, normaliza por estación de origen y reparte las entradas reales observadas como viajes probables a cada destino.
- **[Actualizado 2026-09-27]** Parametrizado por `DIAS_SIMULACION` (lista de fechas, hoy 14 días de enero 2026). Cada fecha se etiqueta `laboral` / `fin_de_semana` según su día de la semana; el fin de semana usa un perfil horario derivado programáticamente del laboral (aplanado de picos + desplazamiento de actividad a mediodía/tarde), no una segunda tabla de constantes inventada. Genera una `matriz_od_sintetica_<fecha>.csv` por día y un `manifiesto_dias_simulados.csv` (fecha, tipo_día, archivo) que `simulador_congestion.py` consume para no duplicar la lista de días entre scripts.

2.3 simulador_congestion.py — Motor de estrés dinámico

- Carga el grafo base y la matriz O-D del día simulado.
- Por cada hora de operación (5–23h), copia el grafo, enruta cada flujo O-D con `nx.shortest_path` (ponderado por `tiempo_minutos`) y acumula `carga_pasajeros` por arista.
- Aplica **función BPR adaptada** para convertir carga en tiempo congestionado:

$$T_c = T_b \times (1 + \alpha \cdot (V/C)^\beta), \alpha=0.15, \beta=4, \text{ tope en } 4 \times T_b$$

[Actualizado 2026-09-27, Sección 6] con capacidad **por sentido y por línea** (trenes/hora × carros × 170 pasajeros; de 12,780 pas/h en L12 a 45,699 en L1) y V/C calculada con la carga del sentido más cargado. Antes: 35,000 pas/h para toda la red contra la carga de ambos sentidos sumada.

- [Actualizado 2026-09-27] **Generador de eventos estocásticos** (reemplaza la inyección determinista de Línea 9): por cada (hora, línea, tipo de evento) del día se sortea ocurrencia de fallas mecánicas e incidentes de plataforma con un proceso de Poisson por (hora, línea) modulado por hora pico, y si ocurre, severidad (Beta(2,5), sesgada a eventos leves), duración (1-4 horas) y tramo afectado. [Sección 6] Las tasas están calibradas con fuentes públicas (desalojos del STC; días con lluvia del SMN). La lluvia es un episodio por día lluvioso que alcanza a las líneas de superficie (A/B/12) de forma correlacionada. La lluvia y la falla mecánica actúan como carga fantasma sobre la BPR; el incidente de plataforma **suspende** el tramo (retraso medio de 30·s² min). Nada de esto se escribe en `carga_pasajeros_red`, que queda como ridership real limpio, y el evento se registra en el dataset vía `hay_evento / tipo_evento / severidad_evento`. Acepta `--semilla` y `--salida`.
- El ruteo ya no recalcula `nx.shortest_path` por cada fila de la matriz O-D y por cada hora: como el peso de ruteo (`tiempo_minutos`) es estático, la ruta de cada par origen-destino se calcula una sola vez y se cachea (necesario para que escalar a 14 días fuera viable en tiempo razonable). Con el recorrido de la matriz O-D por `zip` en vez de `iterrows`, los 14 días corren en ~1.5 min.
- Construye ventanas temporales (t-1, t, t+1) de `congestibilidad` por tramo (sin fugar información entre días) y exporta `dataset_features_entrenamiento.csv`, ahora con **52,360 registros** de 14 días (antes 3,740 de un solo día).

2.4 entrenador_anticipatorio.py — Modelo predictivo

- Ordena cronológicamente y separa train/test 80/20 **sin aleatorizar** (evita fuga de información temporal).
- Features: `hora`, `tiempo_ideal`, `congestibilidad_t`, `congestibilidad_t_minus_1`, `hay_evento`, `severidad_evento` y dummies one-hot de `tipo_evento` (los modelos de sklearn no aceptan texto). [2026-09-27, Sección 5] Ubicación en la red: dummies de `linea` (derivada del ID GTFS del andén; las aristas entre líneas distintas son `transbordo`) y `tramo_congestion_media`, un target encoding suavizado del tramo (congestión futura media del tramo, calculada **solo con entrenamiento**). Target: `target_congestibilidad_t_plus_1`.
- [2026-09-27] Compara 6 configuraciones (`RandomForestRegressor`, `GradientBoostingRegressor`, `HistGradientBoostingRegressor`, cada una con y sin ubicación), con dos evaluaciones: el split cronológico 80/20 y una validación *rolling origin* sobre los últimos 5 días. Reporta MAE/RMSE global y, por separado, en filas con `hay_evento=1`. Elige la configuración con menor RMSE en filas con evento en la validación por días (hoy: `GradientBoosting + linea/tramo`).
- Explicabilidad por **importancia por permutación** (aumento del RMSE al barajar cada variable; las dummies de una misma variable se barajan juntas), agnóstica al modelo, en `importancia_variables.png`.
- Exporta a `modelos/modelo_anticipatorio.pkl` un paquete con el modelo, su codificación (categorías y target encoding) y el inicio del set de prueba; `ruteo_anticipatorio.py` reutiliza `construir_features()` del entrenador para no desincronizarse.

2.5 ruteo_anticipatorio.py — Integración estimación + ruteo

- Carga grafo base + modelo entrenado + dataset de contexto (usado como proxy de "sensores en tiempo real").
- `proyectar_hora(fecha, hora) + grafo_con_retraso(...)`: para cada arista predice el retraso de la hora siguiente y lo suma al tiempo ideal, generando un grafo proyectado. El contexto se filtra por fecha además de hora porque el dataset contiene 14 días, y las features se construyen con la misma función (`construir_features`) y codificación con que se entrenó el modelo.
- [2026-09-27] Los tiempos "reales" se miden contra la congestión simulada de la hora siguiente (`target_congestibilidad_t_plus_1`). Antes se medían contra la propia proyección de la IA, lo que favorecía a la IA por construcción.
- Compara ruta estática (Dijkstra sobre tiempo ideal) vs. ruta anticipatoria (Dijkstra sobre pesos proyectados por IA), evaluando ambas contra el tráfico real proyectado.
- Caso de prueba demostrado: Pantitlán → Auditorio, 7:00 AM (día laboral 2026-01-13). Con el modelo multi-día, ambos sistemas eligen la misma ruta (Línea 9 hasta Tacubaya, transbordo a Línea 7): tiempo ideal 37.94 min, tiempo real en t+1 41.22 min (con el modelo `RandomForest` anterior la proyección era 40.33 min). Ya no se reproduce el desvío que mostraba la versión anterior, porque la falla fija de

Línea 9 (7-9am) que lo provocaba fue reemplazada por eventos estocásticos, y ese día a esa hora no hubo un evento que congestionara esta ruta.

- **[2026-09-27]** Evaluación sistemática sobre todas las horas con evento del set de prueba (ver Sección 5.3).

3. Limitaciones metodológicas actuales (a atender antes de tesis final)

1. ~~Un solo día de simulación~~ **RESUELTO 2026-09-27** (ver [Sección 4](#)): ahora se simulan 14 días (10 laborales + 4 de fin de semana), y el split 80/20 cronológico separa *días* completos (12 de entrenamiento, 3 de prueba, con 1 día compartido en la frontera del corte), no horas dentro del mismo día.
2. ~~Evento de interrupción hardcoded~~ **RESUELTO 2026-09-27**: la falla fija de Línea 9 fue reemplazada por un generador de eventos estocásticos (lluvia, falla mecánica, incidente de plataforma) con tasas tipo Poisson por hora/línea/temporada y severidad/duración muestreadas. Tasas calibradas con fuentes públicas en la Sección 6.3.
3. ~~Capacidad constante para toda la red~~ **RESUELTO 2026-09-27** (Sección 6.1): capacidad por sentido y por línea derivada de datos del STC. Queda como supuesto la frecuencia por línea (derivada, no publicada; subestima el intervalo de la Línea A) y la capacidad de los transbordos (35,000).
4. **Horizonte de predicción discretizado a 1 hora**, mientras el objetivo de la tesis pide horizonte de 10–60 min. Falta granularidad sub-horaria.
5. **Perfiles origen/destino hardcoded** (listas fijas de 10 estaciones): no se derivan de un análisis estadístico de la matriz de afluencia real, sino de un supuesto manual razonado.
6. **[2026-09-27] Eventos raros → poca evidencia** (Secciones 6.4 y 6.5): con tasas calibradas solo 0.08–0.29% de las filas tienen evento. La selección de modelo cambia entre semillas y la ventaja del ruteo anticipatorio en ahorro total no es significativa. Hace falta más simulación (más semillas o más días). Además, lluvia y falla mecánica casi no generan retraso (la carga fantasma apenas mueve la BPR con $\beta=4$); solo la suspensión por incidente lo hace. **Actualización (Sección 7)**: con 30 semillas la selección se fijó (RF + línea/tramo) y las pérdidas del ruteo ya tienen IC, pero la ventaja en ahorro total sigue sin ser significativa (60%): no es un problema de tamaño de muestra sino de que la IA predice el valor esperado de incidentes que duran poco (Sección 7.4).
7. ~~Lluvia independiente por línea y sobreestimada~~ **RESUELTO PARCIALMENTE 2026-09-27** (Sección 6.3): un episodio por día lluvioso, con probabilidad mensual del SMN y correlacionado entre líneas. La hora de inicio (uniforme) y la probabilidad de alcanzar cada línea (0.75) siguen siendo supuestos, pendientes de datos del OH-IIUNAM.

Ninguno de estos puntos invalida el trabajo — son exactamente el tipo de simplificación esperable en una primera iteración de prototipo — pero deben documentarse como alcance y quedar como candidatos directos para la siguiente fase.

4. Extensión multi-día + eventos estocásticos (2026-09-27)

Esta sección documenta la ejecución del plan de la sesión anterior (ver historial de commits). Cambios de código:

- `generador_sintetico_horario.py`: parametrizado por `DIAS_SIMULACION` (14 fechas de enero 2026), perfil horario laboral/fin de semana por `tipo_dia`, una matriz O-D por día y un `manifiesto_dias_simulados.csv` de salida. También se restringió el histórico fuente a solo los días simulados (antes desagregaba las ~1,857 fechas del CSV completo a nivel hora sin usarlas después: un archivo intermedio de ~300 MB para un solo día útil).
- `simulador_congestion.py`: lee el manifiesto y simula los 14 días en un solo run; reemplaza la falla fija de Línea 9 por el generador de eventos estocásticos (Poisson por hora/línea/tipo, severidad Beta(2,5), duración 1-4h); cachea rutas estáticas por par origen-destino (el cuello de botella real al escalar a varios días); agrega `tipo_dia`, `hay_evento`, `tipo_evento`, `severidad_evento` al dataset.
- `entrenador_anticipatorio.py`: si el dataset trae columnas de evento, las agrega como features (`hay_evento`, `severidad_evento`, dummies de `tipo_evento`).
- `ruteo_anticipatorio.py`: **corrección detectada al verificar el pipeline completo**. El script seguía construyendo `X_pred` con las 4 features originales, mientras el modelo reentrenado espera 10, y fallaba con `ValueError: The feature names should match those that were passed during fit`. Además tenía un error silencioso: filtraba el contexto solo por `hora`, así que con 14 días cada arista quedaba con el valor del último día leído, mezclando días distintos. Ahora usa `modelo.feature_names_in_` y filtra por `fecha_viaje + hora_viaje` (caso de prueba: `2026-01-13`, 7:00).

4.1 Dataset resultante

	Versión anterior (1 día, evento determinista)	Versión actual (14 días, eventos estocásticos)
Registros	3,740	52,360
Días distintos	1	14 (10 laborales, 4 fin de semana)
Filas con evento activo	~ desconocido (no etiquetado)	870 (1.66%): 707 lluvia, 94 incidente de plataforma, 69 falla mecánica
Split train/test (80/20 cronológico)	Mismo día en train y test (fuga temporal de facto)	12 días train / 3 días test (1 día compartido en la frontera del corte)
Tiempo de generación (<code>simulador_congestion.py</code>)	segundos	~5.5 min (14 días, con caché de rutas)

4.2 Cambio en MAE / RMSE

Métrica	Baseline (1 día, sin eventos aleatorios)	Nuevo (14 días, eventos estocásticos)	Δ
MAE (min)	0.0000109	0.0028	×256
RMSE (min)	0.000233	0.0313	×134

Lectura correcta de este resultado: el error subió, y eso es lo esperado y lo correcto, no una regresión del modelo. El baseline de 1 día tenía una fuga temporal de facto — el split 80/20 separaba *horas* del mismo día con el mismo patrón de falla recurrente (L9, 7-9am, todos los días idéntico), así que el modelo memorizaba trivialmente el patrón y el error caía a prácticamente cero (MAE ≈ 10⁻⁵ min, una precisión sin sentido físico). Con 14 días y eventos verdaderamente estocásticos (distintos tipos, tramos, horas, severidades y duraciones cada día), el modelo debe generalizar de 12 días vistos a 3 días no vistos con eventos nuevos que nunca ocurrieron exactamente igual en entrenamiento. Un MAE de ~0.003 min y RMSE de ~0.031 min sobre un target cuyo rango va de 0 a 6.09 min (media 0.015, con el 75% de los tramos en 0 — la red solo se congestiona en tramos/horas puntuales) es un desempeño razonable para un primer modelo con features todavía limitadas (no incluye, por ejemplo, la línea/tramo como variable categórica). Esto también resuelve la limitación #1 de la Sección 3: el split ahora separa días reales, no horas del mismo día.

Importancia de variables (nuevo modelo): `congestibilidad_t` (50.3%) y `hora` (41.9%) siguen dominando; el bloque de variables de evento (`severidad_evento`, `hay_evento`, dummies de `tipo_evento`) aporta en conjunto ~1.9% — señal débil pero presente, consistente con que solo 1.66% de las filas tienen evento activo. Ampliar esa señal (más días, o sobremuestrear horas con evento) es candidato directo para la siguiente iteración si se quiere que el modelo dependa más del contexto de eventos y menos de la inercia (`congestibilidad_t`).

4.3 Supuestos de simulación explícitos (para citar en el capítulo de metodología)

- Ventana de 14 días naturales de enero 2026 (única franja con datos reales completos en el CSV fuente); no hay quincena ni variación mensual/estacional real en la muestra, solo la distinción laboral/fin de semana.
- Perfil horario de fin de semana derivado programáticamente del laboral (aplanado + desplazamiento a mediodía/tarde), no calibrado con datos reales de fin de semana.
- Tasas de eventos (Poisson), factores de hora pico/temporada de lluvias y factores de impacto por tipo de evento son **ilustrativos**, no una calibración estadística sobre incidencia real del STC (ver Sección "Análisis de factibilidad técnica" más abajo, que ya anticipaba este punto y sigue vigente para la siguiente iteración).

4.4 Siguiente paso (ejecutado — ver Sección 5)

Bitácora de ingeniería. Punto de partida: el dataset de entrenamiento ya tenía 14 días con eventos estocásticos, pero la señal de evento (`hay_evento` / `tipo_evento` / `severidad_evento`) solo aportaba ~1.9% de importancia porque apenas 1.66% de las filas tenían un evento activo. Antes de calibrar tasas contra fuentes reales se buscó subir la proporción de señal explotable. Pasos implementados:

1. Se añadió `línea` / `tramo` como variable categórica (dummies de línea + target encoding del tramo) para que el modelo pudiera aprender qué tramos son estructuralmente más propensos a congestión; hasta entonces el modelo no sabía en qué parte de la red estaba parado.
2. Se evaluó migrar de `RandomForestRegressor` a `GradientBoostingRegressor` y a `HistGradientBoostingRegressor` (que maneja mejor el desbalance de clases raras), reportando la predicción en las filas con `hay_evento=1` aparte del MAE/RMSE global, ya que ese segmento era <2% de los datos y podía quedar diluido.
3. Se documentó en `avances.md` una propuesta concreta de fuentes para calibrar `TASA_BASE_LLUVIA`, `TASA_BASE_FALLA_MECANICA` y `TASA_BASE_INCIDENTE_PLATAFORMA` en `simulador_congestion.py` (climatología SMN/Conagua para lluvia; boletines de contingencia,

notas de prensa y transparencia del Metro para fallas/incidentes), como aproximación documentada y no como calibración estadística rigurosa (Sección 5.4).

4. En `ruteo_anticipatorio.py`, además del caso fijo (Pantitlán → Auditorio, 2026-01-13 7:00, donde la ruta IA coincidía con la estática), se agregó un recorrido de las fechas/horas del conjunto de prueba con `hay_evento=1` que compara ruta estática vs. anticipatoria en los pares O-D que cruzan los tramos afectados y reporta en cuántos casos la IA cambia la ruta y cuántos minutos ahorra (Sección 5.3).

5. Ubicación en la red, comparación de modelos y evaluación sistemática del ruteo (2026-09-27)

Ejecución del plan de la Sección 4.4. Cambios de código:

- `entrenador_anticipatorio.py`: agrega la ubicación en la red como features (`linea` one-hot + `tramo_congestion_media`, target encoding suavizado calculado solo con entrenamiento); compara RandomForest / GradientBoosting / HistGradientBoosting con y sin ubicación; agrega validación *rolling origin* por días; reporta métricas en filas con evento aparte; cambia la explicabilidad a importancia por permutación; exporta un paquete (`modelos/modelo_anticipatorio.pkl`, reemplaza a `modelo_anticipatorio_rf.pkl`) con modelo + codificación + inicio del set de prueba.
- `ruteo_anticipatorio.py`: reutiliza `construir_features()` del entrenador; mide los tiempos contra la congestión real de $t+1$; agrega la evaluación sistemática sobre horas con evento (Sección 5.3), que guarda el detalle en `datos_procesados/evaluacion_ruteo_eventos.csv`.

5.1 Comparación de modelos

Métricas en minutos de retraso sobre el tiempo ideal. "Evento" = filas con `hay_evento=1`.

Split cronológico 80/20 (prueba: 16–18 de enero; 10,472 filas, **137 con evento**):

Configuración	MAE	RMSE	MAE evento	RMSE evento
HistGradientBoosting (sin ubicación)	0.0030	0.0297	0.0301	0.1236
GradientBoosting (sin ubicación)	0.0032	0.0303	0.0291	0.1263
GradientBoosting + línea/tramo	0.0039	0.0306	0.0309	0.1296
HistGradientBoosting + línea/tramo	0.0036	0.0307	0.0324	0.1346
RandomForest + línea/tramo	0.0027	0.0313	0.0364	0.1496
RandomForest (sin ubicación) — <i>modelo anterior</i>	0.0028	0.0313	0.0363	0.1541

Validación por días (5 pliegues: cada día del 14 al 18 de enero se predice con un modelo entrenado con todos los días previos; **255 filas con evento**):

Configuración	MAE	RMSE	MAE evento	RMSE evento
GradientBoosting + línea/tramo	0.0042	0.0272	0.0369	0.1193
RandomForest + línea/tramo	0.0032	0.0285	0.0488	0.1459
HistGradientBoosting + línea/tramo	0.0044	0.0311	0.0562	0.1728
HistGradientBoosting (sin ubicación)	0.0044	0.0315	0.0591	0.1801
RandomForest (sin ubicación) — <i>modelo anterior</i>	0.0036	0.0321	0.0579	0.1844
GradientBoosting (sin ubicación)	0.0043	0.0385	0.0529	0.1965
<i>Referencia: persistencia (predecir $t+1 = t$)</i>	0.0151	0.0806	0.0491	0.1802

Lectura:

- Las dos evaluaciones no coinciden, y la validación por días es la más confiable.** En el split 80/20 la ubicación parece no ayudar, pero ese set tiene solo 3 días y 137 filas con evento. En la validación por días (5 días, 255 filas con evento) la ubicación mejora a los tres

modelos, y más en el segmento con evento: RandomForest baja su RMSE con evento de 0.184 a 0.146 (−21%) y GradientBoosting de 0.197 a 0.119 (−39%). Por eso el criterio de selección es el RMSE con evento en la validación por días.

2. **Modelo elegido: GradientBoosting + línea/tramo.** Frente al modelo anterior (RandomForest sin ubicación), en la validación por días reduce el RMSE con evento 35% (0.184 → 0.119) y el RMSE global 15% (0.032 → 0.027). A cambio, su MAE global es algo peor (0.0042 vs. 0.0036): predice retrasos pequeños mayores a 0 en tramos que no se congestionan (75% del target es 0). Para el ruteo pesa más acertar los picos de retraso, que es lo que mide el RMSE.

3. **HistGradientBoosting no mejoró sobre GradientBoosting** a pesar de lo planteado en la Sección 4.4. El problema no es tanto el desbalance como la poca señal (punto 5).

4. **Todos los modelos superan a la persistencia** en RMSE global y con evento, así que el modelo aprende algo más que "mañana igual que hoy".

5. **La señal de evento sigue siendo débil.** En la importancia por permutación del modelo elegido (aumento del RMSE al barajar la variable, set de prueba 80/20), dominan congestibilidad_t (0.022 min) y congestibilidad_t_minus_1 (0.017), seguidas de hora (0.003) y tramo_congestion_media (0.003). tipo_evento aporta 0.0005 y hay_evento / severidad_evento ≈ 0. La causa probable está en el simulador, no en el modelo: con capacidad constante y BPR, un evento típico cambia poco el retraso, y ese efecto ya queda reflejado en congestibilidad_t (el evento ya estaba activo en t). Se agrega como limitación #6 en la Sección 3.

5.2 Métricas del modelo exportado

El modelo exportado es GradientBoosting + línea/tramo , entrenado con el 80% inicial (no con todo el dataset, para que el ruteo se evalúe en días que el modelo no vio). En el 20% final: MAE 0.0039 min, RMSE 0.0306 min; en las 137 filas con evento: MAE 0.0309 min, RMSE 0.1296 min.

5.3 Evaluación sistemática del ruteo en horas con evento

Nota: estos resultados usan las tasas sin calibrar y la carga de ambos sentidos sumada. Quedan reemplazados por la Sección 6.5, donde la conclusión cambia.

Diseño. Se recorren las 27 horas con al menos un tramo en evento del set de prueba (16, 17 y 18 de enero). Para cada hora se toman todos los pares origen-destino (una estación por nombre, 163 estaciones) cuya ruta estática cruza un tramo con evento: 84,650 casos. Para cada caso se calculan cuatro rutas y todas se miden contra el tráfico real simulado de la hora siguiente:

- **Estático:** tiempo ideal, sin información de tráfico (baseline).
- **Reactivo:** congestión observada en la hora actual t (sistema reactivo del índice 5.5).
- **Anticipatorio:** congestión que la IA proyecta para t+1.
- **Oráculo:** congestión real de t+1. Es la cota superior: el mayor ahorro posible.

Sistema	Cambia ruta	Casos que ganan / pierden vs. estático	Ahorro medio si cambia	Ahorro neto total	% del ahorro posible capturado
Reactivo	542 (0.64%)	220 / 320	−0.02 min	−8.8 min	−5.0%
Anticipatorio (IA)	282 (0.33%)	148 / 134	+0.40 min	+114.0 min	64.6%
Oráculo	234 (0.28%)	234 / 0	+0.75 min	+176.3 min	100%

Lectura:

1. **Resultado central: el ruteo anticipatorio captura 64.6% del ahorro máximo posible; el reactivo empeora al usuario (−5%).** El sistema reactivo esquiva la congestión de la hora actual, que en la hora siguiente ya cambió (los eventos duran 1–4 h y la demanda cambia cada hora). Por eso cambia de ruta más veces (542) y pierde en más casos de los que gana (320 vs. 220). La IA cambia menos (282) y gana más de lo que pierde: +1.12 min en promedio cuando acierta y −0.39 min cuando falla. Es el argumento cuantitativo para la tesis: anticipar es mejor que reaccionar.
2. **La magnitud absoluta es pequeña.** Solo 0.28% de los casos tiene una ruta mejor que la estática (oráculo), el ahorro máximo en un caso es 1.97 min y el viaje medio dura ~40 min. Esto viene del simulador, no del ruteo: con la capacidad constante de 35,000 pas/h y BPR, un evento casi nunca vuelve una ruta alternativa (con transbordos de 5 min) más rápida que la directa. Para defender el ahorro en tiempo, antes hay que diferenciar capacidades por tramo (limitación #3) o dar a los incidentes un efecto de cierre de tramo (ya previsto en el análisis de factibilidad, tabla 3.1).
3. **La IA se equivoca en casi la mitad de sus cambios de ruta** (134 de 282). Esto es consistente con su RMSE con evento (−0.12–0.13 min): el ahorro disponible por caso es del mismo orden que el error del modelo.

4. **Cautela estadística:** los 84,650 casos no son independientes. Salen de 27 horas y de pocos eventos, y muchos pares O-D comparten los mismos tramos afectados. La conclusión (anticipatorio > estático > reactivo) es consistente en esta muestra, pero para citarla con intervalos de confianza hay que repetir la simulación con varias semillas aleatorias.

5.4 Propuesta de fuentes para calibrar las tasas de eventos

Parámetros en `simulador_congestion.py`: `TASA_BASE_LLUVIA = 0.02`, `TASA_BASE_FALLA_MECANICA = 0.01` y `TASA_BASE_INCIDENTE_PLATAFORMA = 0.006` (eventos esperados por hora y por línea en hora valle), `FACTOR_HORA_PICO = 2.5` (6 horas pico) y `FACTOR_TEMPORADA_LLUVIAS = 6.0` (mayo–octubre). La operación simulada va de las 5 a las 23 h (19 horas: 13 valle + 6 pico), así que el número esperado de eventos por línea y por día es $\lambda_{\text{base}} \times (13 + 2.5 \times 6) = \lambda_{\text{base}} \times 28$.

Chequeo de orden de magnitud con los valores actuales:

Tipo	Eventos esperados/día (tasas actuales)	Referencia real
Falla mecánica + incidente de plataforma (12 líneas)	$(0.01 + 0.006) \times 28 \times 12 \approx 5.4$	3,708 incidentes con desalojo de trenes entre 2018 y agosto de 2022 , según respuesta del STC vía la Plataforma Nacional de Transparencia (El Universal): $\approx 1,704$ días $\rightarrow \approx 2.2$ por día en toda la red. La falla más frecuente fue el sistema de puertas (34 de cada 100).
Lluvia en enero (3 líneas de superficie)	$0.02 \times 28 \times 3 \approx 1.7$	Enero es temporada seca en la CDMX; los días con lluvia del mes se obtienen de las Normales Climatológicas del SMN (observatorio de Tacubaya).
Lluvia en mayo–octubre	$0.02 \times 6 \times 28 \approx 3.4$ por línea	Idem, meses de temporada de lluvias.

Las tasas de falla/incidente están en el orden correcto (unas 2.5 veces la referencia). Parte de la diferencia es esperable, porque los desalojos son solo los incidentes más graves y la referencia subestima el total. La lluvia de enero parece sobreestimada.

Fuentes propuestas y método de calibración:

- Lluvia: frecuencia diaria.** Normales Climatológicas del SMN/Conagua (<https://smn.conagua.gob.mx/es/climatologia/informacion-climatologica/normales-climatologicas-por-estado>), Ciudad de México, estación Tacubaya (observatorio de referencia del SMN): número medio de días con precipitación por mes. Con eso, $P(\text{lluvia en el día} \mid \text{mes})$ sustituye al factor binario de temporada (`FACTOR_TEMPORADA_LLUVIAS`) por un valor para cada mes.
- Lluvia: distribución horaria y extensión espacial.** Red pluviométrica del Observatorio Hidrológico del Instituto de Ingeniería UNAM (OH-IIUNAM), con unas 55 estaciones en la CDMX que registran la intensidad minuto a minuto (<https://www.iingen.unam.mx/es-mx/AlmacenDigital/Notas/Paginas/observatoriohidrologico.aspx>). Con las estaciones cercanas a las líneas A, B y 12 se estima (a) la probabilidad de lluvia por hora del día, que en la CDMX se concentra en la tarde-noche y no sigue la hora pico del Metro (hoy se le aplica `FACTOR_HORA_PICO` sin justificación física), y (b) con qué frecuencia llueve a la vez en varias líneas, para dejar de sortear cada línea por separado (limitación #7). Hay que pedir los datos al OH-IIUNAM; su publicación en tiempo real es por redes y web.
- Fallas mecánicas e incidentes de plataforma.**
 - Punto de partida documentado: la cifra de 3,708 desalojos 2018–2022 citada arriba (El Universal, <https://www.eluniversal.com.mx/metropoli/de-2018-2022-se-reportaron-3-mil-708-incidentes>), que da $\lambda_{\text{base}} \approx 2.2 / (12 \text{ líneas} \times 28) \approx 0.0065$ por línea-hora valle para ambos tipos juntos, suponiendo el mismo factor de hora pico.
 - Para separarlo **por línea, tipo y hora**: solicitud de información al STC por la Plataforma Nacional de Transparencia o su Unidad de Transparencia (<https://www.metro.cdmx.gob.mx/transparencias/transparencia-cdmx>), pidiendo el registro de incidentes/averías con fecha, hora, línea, estación y causa. Hay antecedentes de que el STC entrega este tipo de datos: la cifra anterior y el reporte de 136 averías en la Línea 12 tras su reapertura (El Gráfico, 2024).
 - Complemento (y validación del factor de hora pico): recopilación manual de los avisos de la cuenta oficial del Metro en redes sociales durante algunas semanas.
- Método de conversión a tasa de Poisson** (para cualquiera de las fuentes): si en D días se observaron N eventos de un tipo en una línea, con F = factor de hora pico, entonces $\lambda_{\text{base}} = N / (D \times (13 + 6 \cdot F))$. Si la fuente trae la hora del evento, F también se estima: la tasa por hora en hora pico dividida entre la tasa por hora en valle.

Esto es una **aproximación documentada, no una calibración estadística rigurosa**: los desalojos no son todos los incidentes, la cifra agrega todas las líneas y todo el periodo 2018–2022, y la climatología de Tacubaya representa un solo punto de la ciudad. Aun así, cada parámetro queda respaldado por una fuente citable en lugar de un valor ilustrativo.

5.5 Siguiente paso (ejecutado — ver Sección 6)

Bitácora de ingeniería. Punto de partida: la evaluación sistemática (Sección 5.3) mostraba que el ruteo anticipatorio capturaba 64.6% del ahorro posible y el reactivo empeoraba (-5%), pero el ahorro absoluto era pequeño (el oráculo solo mejoraba 0.28% de los casos, máximo 1.97 min) porque, con capacidad constante y BPR, los eventos casi no cambiaban la ruta óptima; además el modelo apenas usaba las variables de evento. Se atacó la causa en el simulador. Pasos implementados:

1. Se sustituyó `CAPACIDAD_PROMEDIO_TRAMO_HORA` por una capacidad por línea (trenes por hora × capacidad del tren, con valores documentados del STC: parque vehicular, disponibilidad e intervalo mínimo) (Sección 6.1).
2. Se hizo que `incidente_plataforma` suspenda temporalmente el tramo en lugar de solo agregar carga fantasma (Sección 6.2).
3. Se aplicaron las tasas de la Sección 5.4 ($\lambda_{base} \approx 0.0065$ combinado para falla+incidente, lluvia por mes según días con precipitación de Tacubaya, sin factor de hora pico para lluvia, y lluvia correlacionada entre las líneas A, B y 12) (Sección 6.3).
4. Se regeneró el dataset, se reentrenó (el entrenador ya comparaba modelos) y se volvió a correr `ruteo_anticipatorio.py`. La simulación se repitió con 5 semillas para reportar la media y el intervalo del % de ahorro capturado por cada sistema (Secciones 6.4 y 6.5), y se actualizó `avances.md`.

6. Capacidad por línea, suspensión de tramos, tasas calibradas y experimento multi-semilla (2026-09-27)

Ejecución del plan de la Sección 5.5. Cambios de código:

- `simulador_congestion.py`:
 - Capacidad por sentido y por línea en lugar de la constante de 35,000 pas/h.
 - La V/C se calcula con la carga del **sentido más cargado**.
 - `incidente_plataforma` suspende el tramo.
 - Tasas de eventos calibradas con fuentes públicas; la lluvia es un solo episodio por ciudad.
 - Argumentos `--semilla` y `--salida`.
 - Cambió `iterrows` por `zip`: 14 días en ~1.5 min en vez de ~5.3 min. La carga de pasajeros resultante es idéntica fila por fila a la de la versión anterior.
- `entrenador_anticipatorio.py` y `ruteo_anticipatorio.py`: argumentos de rutas de entrada/salida (los valores por defecto son los de siempre). El ruteo guarda además un resumen (`datos_procesados/resumen_ruteo_eventos.csv`).
- `experimento_semillas.py` (nuevo): corre simulador → entrenador → ruteo con 5 semillas (42, 7, 13, 101, 2026), 3 en paralelo, en `datos_procesados/semillas/semilla_<s>/`, y agrega los resultados (`--solo-agregar` recalcula sin volver a simular).

6.1 Capacidad por línea

Capacidad por sentido = trenes/hora × carros por tren × 170 pasajeros por carro.

Dato	Valor	Fuente
Capacidad por tren	6 carros = 1,020 pas.; 9 carros = 1,530 → 170 por carro	STC, Parque Vehicular
Trenes asignados por línea	L1 50, L2 41, L3 54, L4 14, L5 25, L6 15, L7 32, L8 30, L9 34, LA 33, LB 36, L12 30	STC, Parque Vehicular
Carros por tren	6 en L4 y L6 (29 trenes neumáticos de 6 carros = 14 + 15); 7 en L12 (30 férreos de 7 carros); LA: 22 de 9 + 11 de 6 → promedio 8; resto 9	Deducido del desglose del Parque Vehicular
Disponibilidad	256 de 394 trenes en servicio (64.9%)	STC vía transparencia, Expansión, nov. 2024
Intervalo mínimo	2 min en hora pico	STC, Preguntas frecuentes

El STC no publica la frecuencia por línea, así que se deriva: `trenes/hora = trenes de la línea × 0.649 × 60 / (2 × tiempo de recorrido GTFS)`, con tope en 30 trenes/hora (intervalo de 2 min). No se incluye tiempo de maniobra en terminales porque no está documentado.

Línea	12	6	4	B	8	5	2	7	A	3	9	1
Capacidad por sentido (pas/h)	12,780	16,830	17,539	28,003	28,973	30,758	31,876	34,653	35,707	41,432	43,370	45,699

Línea	12	6	4	B	8	5	2	7	A	3	9	1
Intervalo derivado (min)	5.6	3.6	3.5	3.3	3.2	3.0	2.9	2.7	2.3	2.2	2.1	2.0

Contraste con intervalos de hora pico reportados por un sitio no oficial ([metrocdmx.net](#)): L1 ≈ 2 min (derivado 2.0 ✓), L3 2–3 (2.2 ✓), L12 3–5 (5.6, algo más largo), LA 4–6 (2.3, **subestimado**). Para la Línea A la capacidad derivada probablemente es demasiado alta; la causa más probable es que su disponibilidad real sea menor que el promedio del sistema.

Carga por sentido. El grafo es no dirigido y antes la carga de un tramo sumaba los dos sentidos, que se comparaba contra una capacidad de un solo sentido. Ahora se lleva la carga de cada sentido (las rutas del simulador son secuencias ordenadas) y la V/C usa la del sentido más cargado. `carga_pasajeros_red` en el dataset sigue siendo la suma de ambos sentidos. Los pasillos de transbordo conservan la capacidad anterior (35,000), porque no hay dato.

Efecto: la congestión se concentra en las líneas de menor capacidad (12, 4, 6, B, 8). La Línea 9 a las 7:00 deja de congestionarse, y en el caso de prueba Pantitlán → Auditorio el tiempo real queda en 37.96 min, frente a 37.94 ideales.

6.2 Suspensión de tramos por incidente de plataforma

`incidente_plataforma` ya no suma carga fantasma: suspende el tramo. La severidad `s` se interpreta como la fracción de la hora con el tramo cerrado. Un pasajero que llega durante el cierre espera en promedio $60 \cdot s / 2$ min, y llega durante el cierre una fracción `s` de los pasajeros. Por eso el retraso medio de la hora es **`30 · s²` min** (hasta 30 min si el tramo pasa la hora completa cerrado) y se suma después del BPR. La lluvia y la falla mecánica siguen como carga fantasma, ahora proporcional a la capacidad de la línea afectada.

Con la semilla 42, las 27 filas con incidente tienen congestión media de 3.3 min (máximo 7.7). La falla mecánica queda en 0.18 min y la lluvia en 0.003. Con capacidad por sentido y BPR con $\beta=4$, la carga fantasma casi no mueve el tiempo de viaje; solo la suspensión produce retrasos grandes.

6.3 Tasas de eventos calibradas

Parámetro	Antes	Ahora	Fuente / cálculo
Falla mecánica + incidente de plataforma	0.016 por línea-hora valle (≈ 5.4 eventos/día en la red)	0.0065 (≈ 2.2/día)	3,708 desalojos 2018–ago 2022 (Sección 5.4). Reparto entre tipos sin fuente: se conserva la proporción 5:3 → 0.0041 / 0.0024
Factor hora pico (fallas/incidentes)	2.5	2.5	Sin cambio: no hay dato horario (pendiente de solicitud PNT)
Lluvia: probabilidad diaria	Poisson 0.02 por línea-hora, ×6 may–oct	días con lluvia del mes / días del mes: enero 2.8/31 = 9%	Normal climatológica 1981–2010, estación 9048 Tacubaya Central (SMN) : días con lluvia ENE 2.8, FEB 2.2, MAR 3.6, ABR 7.2, MAY 11.5, JUN 17.6, JUL 22.0, AGO 21.0, SEP 17.9, OCT 9.9, NOV 3.0, DIC 1.5
Lluvia: hora	Poisson con factor de hora pico	Un episodio por día lluvioso, hora de inicio uniforme	Sin factor de hora pico: la lluvia no depende de la afluencia. Distribución horaria real pendiente (OH-IIUNAM)
Lluvia: extensión	Sorteo independiente por línea	Un episodio afecta a cada línea de superficie con probabilidad 0.75 (al menos una)	0.75 es ilustrativo; pendiente de correlación espacial OH-IIUNAM

Efecto en el dataset (semilla 42): de 102 eventos y 870 filas con evento (1.66%) se pasa a **40 eventos y 130 filas (0.25%)**. En las 5 semillas hay entre 25 y 40 eventos y entre 0.08% y 0.29% de filas con evento. En enero real, los eventos que afectan al servicio son raros.

6.4 Resultados multi-semilla: modelos

Con eventos raros, cada semilla deja solo 13–25 filas con evento en la validación por días. La selección de modelo por semilla es inestable: **se eligió un modelo distinto en 4 de las 5 semillas** (42: HistGB sin ubicación; 7 y 2026: GradientBoosting + línea/tramo; 13: RandomForest + línea/tramo; 101: GradientBoosting sin ubicación).

Validación por días agregada sobre las 5 semillas (89 filas con evento en total):

Configuración	RMSE global	MAE evento	RMSE evento (agregado)
RandomForest + línea/tramo	0.0666	0.340	1.078
RandomForest (sin ubicación)	0.0670	0.379	1.088
HistGradientBoosting + línea/tramo	0.0906	0.417	1.117
GradientBoosting (sin ubicación)	0.0672	0.371	1.173
GradientBoosting + línea/tramo	0.0684	0.344	1.245
HistGradientBoosting (sin ubicación)	0.1055	0.497	1.321

Lectura: con el simulador recalibrado, la conclusión de la Sección 5.1 (GradientBoosting

- línea/tramo) **no se sostiene**. Agregando semillas gana RandomForest + línea/tramo en RMSE global y con evento, aunque las diferencias entre las dos variantes de RF son pequeñas. El criterio de selección por semilla (RMSE con evento con ~20 filas) es ruido: conviene fijar el modelo con el resultado agregado o elegir por RMSE global (ver 6.6). El modelo exportado para la semilla 42 (el del pipeline por defecto) es HistGB sin ubicación, el peor en el agregado. Su importancia por permutación da peso negativo a `severidad_evento` (-0.013: bajarla *mejora* el error), señal de sobreajuste a los pocos eventos.

6.5 Resultados multi-semilla: ruteo

Se juntan los casos de las 5 semillas: 39 horas con evento en días no vistos y 82,784 casos O-D. El % del ahorro posible **no se promedia por semilla**: cuando el oráculo casi no puede ahorrar, el cociente explota (semilla 101: -2,880%) o queda indefinido (semilla 7: ahorro posible 0). El intervalo de confianza sale de un **bootstrap por conglomerados** (5,000 repeticiones) que remuestrea horas con evento completas, porque los pares O-D de una misma hora comparten tramos afectados y no son independientes.

Sistema	Ahorro total (min)	Pérdidas totales (min)	Cambios de ruta	% del ahorro posible [IC 95%]
Reactivo	5,567	-11,070	9,587	33.4 [-4,305; 97.0]
Anticipatorio (IA)	6,400	-598	2,261	38.5 [-176; 72.3]
Oráculo	16,644	0	5,112	100

Por semilla (ahorro total, min):

Semilla	Horas con evento	Reactivo	Anticipatorio	Oráculo
42	12	8,958	6,916	16,556
7	6	-1,597	-73	0
13	10	-30	13	26
101	10	-1,764	-455	61
2026	1	0.1	0.1	0.1

Lectura:

- No se puede afirmar que la IA ahorre más que el reactivo.** Diferencia puntual: +833 min a favor de la IA, pero la probabilidad bootstrap de que la IA ahorre más es solo **54%**. Solo 12 de las 39 horas tienen ahorro posible > 0.5 min, y el total depende casi por completo de un incidente de la semilla 42 (13–15 h del 16 de enero).
- Resultado robusto: la IA es mucho más conservadora y pierde 18 veces menos.** El reactivo cambia de ruta 4 veces más y acumula -11,070 min en casos donde empeora al usuario, contra -598 de la IA. Esto se repite en las 3 semillas con pérdidas (42, 7, 101). El mecanismo se ve en el incidente de la semilla 42:
 - 13 h: el reactivo capta el 100% del ahorro (8,026 min) y la IA 74% (5,958).
 - 14 h: el incidente continúa; el reactivo vuelve a captar el 100% (8,026) y la IA solo 845.
 - 15 h: **el tramo reabre**. El reactivo sigue esquivándolo y pierde 7,299 min; la IA pierde 36.

Es decir, el reactivo acierta mientras la disrupción persiste y falla al terminar. La IA hace lo contrario: anticipa el final pero subestima la persistencia.

3. Para defender en la tesis que "anticipar es mejor que reaccionar" hacen falta muchas más horas con incidentes de impacto. Con 5 semillas × 3 días de prueba hay unos pocos incidentes relevantes. La conclusión de la Sección 5.3 (IA 64.6% vs. reactivo -5%) dependía de las tasas sobreestimadas y de la carga sumando ambos sentidos, y **queda reemplazada** por esta.

6.6 Siguiente paso (ejecutado — ver [Sección 7](#))

Bitácora de ingeniería. Punto de partida: con el simulador recalibrado (Sección 6), el ruteo anticipatorio perdía 18 veces menos que el reactivo, pero la diferencia en ahorro total no era significativa (probabilidad bootstrap 54%) porque solo había ~12 horas con incidentes de impacto en 5 semillas, y la selección de modelo cambiaba de ganador en 4 de 5 semillas. Pasos implementados (resultados en la [Sección 7](#)):

1. Se cambió el criterio de selección en `entrenador_anticipatorio.py` para que no dependiera de ~20 filas con evento: se fijó la configuración con el resultado agregado multi-semilla (RandomForest + línea/tramo, constante `MODELO_ELEGIDO`) y se justificó en `avances.md` (Sección 7.1).
2. Se aumentó la evidencia: `SEMILLAS` en `experimento_semillas.py` pasó de 5 a 30 (las 5 originales + 1001–1025; 36.7 min con 3 corridas en paralelo) y se volvió a reportar el bootstrap por horas con evento, incluida la probabilidad de que el anticipatorio ahorre más que el reactivo y un IC para las pérdidas de cada sistema (Sección 7.3).
3. Se agregó al modelo la variable `edad_evento` (horas desde que empezó el evento; el simulador conoce `hora_inicio` y `duracion`, pero solo expone la hora de inicio, que sería observable en tiempo real) para que la IA pudiera aprender la persistencia de los incidentes, que era donde perdía contra el reactivo (Sección 7.4).
4. Se actualizó `avances.md` con los resultados.

7. Selección fija de modelo, 30 semillas y edad del evento (2026-09-27)

Ejecución del plan de la Sección 6.6. Cambios de código:

- `simulador_congestion.py`: guarda `edad_evento` por tramo y hora (`hora - hora_inicio` del evento más severo del tramo; 0 en la primera hora, -1 sin evento). La duración no se expone. Los sorteos aleatorios no cambian: para una misma semilla el dataset es idéntico al de la Sección 6 salvo la columna nueva.
- `entrenador_anticipatorio.py`: `edad_evento` entra en `FEATURES_BASE`; el modelo exportado es fijo (`MODELO_ELEGIDO = ('RandomForest', True)`). La comparación de las 6 configuraciones se sigue calculando y guardando, e imprime qué habrían elegido los criterios por semilla (menor RMSE con evento y menor RMSE global), solo como auditoría.
- `experimento_semillas.py`: 30 semillas; el bootstrap reporta IC 95% del ahorro y de las pérdidas de cada sistema, de la diferencia de ahorro y de pérdidas entre anticipatorio y reactivo, y las probabilidades bootstrap de que el anticipatorio ahorre más y pierda menos. Se guarda en `modelos/resultados_semillas_bootstrap.csv`. La estabilidad de la selección se reporta como conteo de ganadores por semilla.
- Se regeneró el pipeline por defecto (semilla 42): dataset, `modelos/modelo_anticipatorio.pkl` (ahora RandomForest + línea/tramo) e `importancia_variables.png`.

7.1 Criterio de selección del modelo

Decisión: el modelo exportado es siempre **RandomForest + línea/tramo**; ya no se elige por semilla.

Justificación. El criterio anterior (menor RMSE con evento en la validación por días) se decidía con 13–25 filas con evento por semilla, y con 5 semillas eligió 4 modelos distintos (Sección 6.4). Con 30 semillas la inestabilidad se confirma: el ganador por RMSE con evento se reparte entre las 6 configuraciones (HistGB sin ubicación 8, GB + línea/tramo 6, HistGB + línea/tramo 6, RF + línea/tramo 4, GB sin ubicación 4, RF sin ubicación 2). Un criterio que elige casi al azar no sirve para el capítulo de metodología. Se fija la configuración que ganó en el agregado de la Sección 6.4, y la comparación se sigue reportando para auditarla.

Contraste con 30 semillas (validación por días agregada; 933 filas con evento, contra 89 con 5 semillas):

Configuración	RMSE global (media por semilla)	MAE global	MAE evento	RMSE evento (agregado)
HistGradientBoosting + línea/tramo	0.0963	0.0063	0.489	1.085
HistGradientBoosting (sin ubicación)	0.1054	0.0067	0.512	1.099
RandomForest + línea/tramo (elegido)	0.0743	0.0034	0.511	1.179
RandomForest (sin ubicación)	0.0735	0.0034	0.522	1.193
GradientBoosting + línea/tramo	0.0743	0.0043	0.488	1.230

Configuración	RMSE global (media por semilla)	MAE global	MAE evento	RMSE evento (agregado)
GradientBoosting (sin ubicación)	0.0740	0.0042	0.515	1.265

Ganador por semilla con menor RMSE global: RF sin ubicación 14, GB sin ubicación 5, GB + línea/tramo 5, RF + línea/tramo 3, HistGB + línea/tramo 3.

Lectura:

- La conclusión de la Sección 6.4 tampoco se sostiene del todo con 30 semillas.** En el RMSE con evento ahora gana HistGB + línea/tramo (1.085 vs. 1.179 del elegido, -8%), pero a cambio de un RMSE global 30% peor (0.096 vs. 0.074) y casi el doble de MAE global, error que viene de las filas sin evento (99.8% del dataset). El orden entre familias cambia según la métrica y las diferencias dentro de cada familia son pequeñas.
- Se mantiene RandomForest + línea/tramo** porque queda en el grupo de menor error global (a 1% de RF sin ubicación, diferencia menor que la variación entre semillas), tiene el menor MAE y es el mejor de ese grupo en RMSE con evento. Elegir por RMSE global puro daría RF sin ubicación, prácticamente equivalente (+1% RMSE con evento).
- Queda documentado como decisión abierta:** si el objetivo del prototipo es solo rutear en horas con evento, HistGB + línea/tramo es candidato. El experimento de ruteo con 30 semillas se hizo con RF + línea/tramo; no se repitió con HistGB.

7.2 Métricas del modelo exportado (semilla 42)

RandomForest + línea/tramo con `edad_evento` , entrenado con el 80% inicial. En el 20% final: MAE 0.0047 min, RMSE 0.1209 min; en las 13 filas con evento: MAE 1.325 min, RMSE 2.564 min. Importancia por permutación: `congestibilidad_t` 0.0436, `tramo_congestion_media` 0.0037, `linea` 0.0014, `severidad_evento` 0.0012, `congestibilidad_t_minus_1` 0.0005, `hora` 0.0002, `edad_evento` 0.0001; el resto ≈ 0. El caso de prueba Pantitlán → Auditorio (2026-01-13, 7:00) no cambia: la IA mantiene la ruta estática (37.96 min reales).

7.3 Resultados con 30 semillas: ruteo

258 horas con evento en días no vistos, **525,446 casos O-D** (antes 39 horas y 82,784 casos). Solo **53 horas** tienen ahorro posible > 0.5 min. Bootstrap por conglomerados (5,000 repeticiones sobre horas con evento completas):

Sistema	Ahorro total (min) [IC 95%]	Pérdidas totales (min) [IC 95%]	Cambios de ruta	% del ahorro posible [IC 95%]
Reactivo	20,154 [-16,622; 57,996]	-29,024 [-50,965; -12,320]	33,953	40.9 [-66.0; 78.8]
Anticipatorio (IA)	23,494 [6,434; 43,875]	-5,483 [-9,744; -2,433]	16,233	47.7 [23.2; 67.6]
Oráculo	49,224 [21,000; 81,549]	0	19,527	100

Comparación anticipatorio – reactivo	Puntual	IC 95%	Probabilidad bootstrap
Diferencia de ahorro total	+3,340 min	[-21,618; 28,904]	60% de que la IA ahorre más
Diferencia de pérdidas	+23,542 min	[7,993; 44,024]	100% de que la IA pierda menos

Por semilla, la IA ahorra más que el reactivo en 18 de 30. En las 5 semillas originales los totales casi no cambian respecto a la Sección 6.5 (IA 6,307 vs. 6,400 min; pérdidas -577 vs. -598), y las 25 semillas nuevas repiten el patrón (IA 17,187 vs. reactivo 14,587; pérdidas -4,906 vs. -17,955).

Lectura:

- Resultado robusto, ahora con IC: la IA pierde 5.3 veces menos que el reactivo** (-5,483 vs. -29,024 min) y el IC de la diferencia excluye el 0. Con más semillas el cociente baja de 18× a 5.3×: el 18× de la Sección 6.5 venía de pocas horas.
- La IA es el único sistema con % de ahorro capturado significativamente positivo** (47.7%, IC [23.2; 67.6]). El IC del reactivo incluye valores muy negativos: según qué horas toquen, empeora al usuario en neto.
- La ventaja en ahorro total sigue sin ser significativa** (probabilidad 60%, antes 54%). Sextuplicar la muestra no la resolvió; la causa es la estructura, no el tamaño:
 - En las 53 horas con ahorro posible, el **reactivo gana en 38**, la IA en 9 (6 empates): mientras la disrupción persiste, reaccionar capta más (37,044 vs. 28,010 min).
 - En las 205 horas sin ahorro posible (el incidente ya terminó o no abre alternativa), el reactivo pierde -16,889 min y la IA -4,516.

El balance neto depende de cuántas horas de "incidente que sigue" contra "incidente que acaba" caigan en la muestra, y eso varía mucho entre semillas.

4. **Para la tesis:** lo defendible es "anticipar reduce el riesgo" (menos pérdidas, % capturado con IC positivo), no "anticipar ahorra más minutos que reaccionar".

7.4 Edad del evento: efecto nulo

Ablación sobre los 30 datasets: RandomForest + línea/tramo en validación por días, con y sin `edad_evento` (mismos pliegues):

Segmento	Filas	RMSE sin edad	RMSE con edad
Global	561,000	0.0859	0.0859
Con evento	933	1.1790	1.1789
Incidente de plataforma	194	2.581	2.581
Incidente, primera hora (edad 0)	104	2.763	2.757
Incidente, edad ≥ 1	90	2.354	2.362

Persistencia observada de los incidentes y predicción media del modelo:

Edad (h)	Filas	Sigue activo en t+1	Retraso real medio en t+1	Predicción media sin edad	Predicción media con edad
0	104	39%	1.85	1.86	1.87
1	58	29%	1.33	0.97	1.03
2	27	15%	0.35	0.61	0.61
3	5	0%	0.00	0.21	0.23

("Sigue activo" = retraso real en t+1 > 1 min.)

Lectura:

1. **La variable no aporta nada** (importancia 0.0001; RMSE idéntico). La información ya estaba en el modelo: la predicción baja con la edad aun sin `edad_evento`, porque `congestibilidad_t_minus_1` distingue un incidente que acaba de empezar (t-1 sin retraso) de uno que ya llevaba una hora. Las gradaciones restantes (edad 1 vs. 2 vs. 3) tendrían que aprenderse de solo ~19 filas de incidente por semilla.
2. **La hipótesis de la Sección 6.6 era incorrecta.** La IA no pierde contra el reactivo por no saber cuánto lleva el incidente: su predicción media ya está calibrada (1.87 vs. 1.85 en la primera hora). Pierde porque predice el **valor esperado**: con 39% de probabilidad de que el incidente siga, el retraso esperado rara vez supera el costo del desvío, y la IA no cambia de ruta. El reactivo supone continuidad al 100%: gana cuando el incidente sigue y pierde cuando acaba. Para el usuario promedio el valor esperado es el criterio correcto (por eso pierde menos). Capturar más ahorro requeriría información que el simulador no expone (duración anunciada del cierre) o un criterio de decisión distinto del valor esperado.
3. Se conserva `edad_evento` en el modelo: no empeora y sería relevante con duraciones más largas o datos reales de incidentes.

7.5 Siguiendo paso

Plan. La Sección 7 dejó dos preguntas abiertas: (a) si la IA pierde ahorro frente al reactivo por falta de información sobre la duración de los incidentes (Sección 7.4), y (b) si HistGB + línea/tramo rutea mejor que el modelo fijo (Sección 7.1). Ambas se responden con el mismo experimento de 30 semillas, así que se atienden juntas. Las limitaciones #4 y #5 de la Sección 3 quedan para la fase siguiente (punto 5).

1. **Aviso de tiempo estimado de restablecimiento** (`simulador_congestion.py`). Cuando hay un incidente o una falla, el STC anuncia un tiempo estimado de restablecimiento; el simulador tiene la duración real pero no la expone. Se agregará la columna `horas_restantes_anunciadas` (-1 sin evento; la lluvia no la trae):
 - o Restante real en $t = \text{hora_inicio} + \text{duracion} - \text{hora} - 1$ (0 = el evento termina al cerrar esta hora).
 - o $\text{Aviso} = \text{round}(\text{restante_real} \times \exp(\epsilon))$, con $\epsilon \sim N(0, \sigma)$ y mínimo 0. σ se controla con un argumento `--ruido-aviso` (0 = aviso perfecto).
 - o El ruido se sortea con un generador aparte (`default_rng(semilla + 1_000_000)`) para no alterar la secuencia de eventos: con la misma semilla, el dataset debe ser idéntico al de la Sección 7 salvo la columna nueva. Se verificará comparando las demás columnas.

2. **Feature en el entrenador** (`entrenador_anticipatorio.py`). `horas_restantes_anunciadas` entra en `FEATURES_BASE` solo si está en el dataset, para que los datasets de la Sección 7 sigan siendo entrenables. Se agregará un argumento `--configuracion` que sobrescriba `MODELO_ELEGIDO` (p. ej. `HistGradientBoosting+geo`), sin cambiar el valor por defecto.
3. **Experimento** (`experimento_semillas.py`). Se agregarán:
- Un argumento `--variante <nombre>` que escriba en `datos_procesados/semillas/<variante>/semilla_<s>/` y pase al simulador y al entrenador los argumentos de esa variante, para no sobrescribir los resultados de la Sección 7.
 - Un modo `--reusar-dataset` que omita el simulador si el dataset ya existe (las variantes que solo cambian el modelo no necesitan resimular; ahorra ~1.5 min por semilla).
 - Una comparación **pareada** entre dos variantes: como las horas con evento son las mismas (mismas semillas), el bootstrap remuestra horas y calcula la diferencia de ahorro y de pérdidas entre variantes en cada hora. Da un IC más estrecho que comparar los IC por separado.

Variantes a correr (misma lista de 30 semillas):

Variante	Simulador	Modelo	Qué responde	Costo aprox.
base	Sección 7 (ya existe)	RF + línea/tramo	Referencia	0
hgb	reusa base	HistGB + línea/tramo	Decisión abierta 7.1	~25 min
aviso_perfecto	--ruido-aviso 0	RF + línea/tramo	Cota: cuánto vale saber la duración exacta	~37 min
aviso_ruidoso	--ruido-aviso 0.5	RF + línea/tramo	Valor de un aviso realista	~37 min

$\sigma = 0.5$ significa que, antes de redondear, el aviso queda entre 0.6 y 1.65 veces el restante real en ~2 de cada 3 casos. Como el ruido es multiplicativo, un evento que termina en esta hora (restante 0) siempre se anuncia bien. Es un supuesto ilustrativo: no hay datos del STC sobre la precisión de sus avisos.

4. **Qué se reporta en `avances.md`** (nueva Sección 8), para cada variante contra `base`: ahorro total, pérdidas, % del ahorro posible capturado (con IC), diferencia pareada con IC y probabilidad bootstrap; y lo mismo contra el reactivo. Criterios de lectura:
- Si `aviso_perfecto` no mejora a `base`, la hipótesis de la Sección 7.4 (la IA pierde por predecir el valor esperado sin saber la duración) es falsa y hay que buscar la causa en otro lado (p. ej. el costo del desvío o el umbral de decisión).
 - Si `aviso_perfecto` mejora pero `aviso_ruidoso` no, el valor del aviso depende de su precisión, y así se reporta.
 - Si `hgb` no gana a `base` en ahorro pareado ni en pérdidas, se cierra la Sección 7.1 manteniendo Random Forest + línea/tramo.
 - Solo si `aviso_ruidoso` hace que la IA supere al reactivo en ahorro total con probabilidad bootstrap $\geq 95\%$, se actualiza la afirmación de la tesis de "anticipar reduce el riesgo" a "anticipar ahorra más".
5. **Fase siguiente (fuera de este plan)**: limitación #4 (horizonte de 10–60 min, que requiere pasos sub-horarios en el simulador) y #5 (perfiles O-D derivados de la matriz de afluencia real). Ambas cambian el dataset de raíz e invalidarían la comparación entre variantes, por eso van después.

Tiempo total estimado: ~1 h 40 min de cómputo (3 corridas en paralelo) más el análisis.

Análisis de factibilidad técnica: eventos estocásticos (lluvia, contingencias/accidentes)

Nota (2026-09-27): el análisis de esta sección ya fue implementado — ver [Sección 4](#). Se conserva completo porque documenta el razonamiento y las fuentes consideradas para calibrar las tasas de eventos, que sigue siendo trabajo pendiente (Sección 4.4).

Conclusión general: es técnicamente factible y compatible con la arquitectura existente, con esfuerzo moderado. El diseño actual ya separa la "capa de infraestructura" (grafo estático), la "capa de demanda" (matriz O-D) y la "capa de estrés" (función BPR + inyección de carga), lo que da puntos de extensión naturales para insertar estocasticidad sin rediseñar el sistema desde cero.

3.1 Dónde se integra cada tipo de evento

Evento	Efecto físico a modelar	Punto de integración en el código actual
Lluvia	Reduce velocidad de acceso a estaciones de superficie/elevadas (Línea A, B, 12 en tramos), incrementa afluencia por trasvase	Multiplicador probabilístico sobre <code>peso_atractividad</code> / <code>pasajeros_viaje</code> en <code>generador_sintetico_horario.py</code> , y/o

Evento	Efecto físico a modelar	Punto de integración en el código actual
	modal (gente que deja camión/bici por metro)	reducción de <code>CAPACIDAD_PROMEDIO_TRAMO_HORA</code> en tramos de superficie dentro de <code>simulador_congestion.py</code>
Falla mecánica / contingencia de línea	Reduce o anula capacidad de un tramo específico por tiempo variable	Ya existe el mecanismo (inyección de carga fantasma en <code>simulador_congestion.py</code>); solo falta parametrizarlo estocásticamente en vez de hardcodearlo
Accidente/incidente de plataforma	Cierre temporal de estación/tramo (grafo con disponibilidad reducida, no solo más lento)	Requiere nueva función que remueva temporalmente una arista o nodo del grafo (<code>G_hora.remove_edge(...)</code>) en vez de solo penalizar el peso — el grafo actual no maneja indisponibilidad total, solo congestión

3.2 Requisitos técnicos adicionales

- 1. Generador de eventos por proceso estocástico.** Un proceso de Poisson (o Binomial Negativa si se quiere sobredispersión) por hora/línea es suficiente y barato computacionalmente: para cada hora simulada, se sortea si ocurre un evento y, si ocurre, se sortea tipo, tramo afectado, severidad y duración. Esto es perfectamente compatible con el bucle `for hora in horas_operacion` que ya existe en `simulador_congestion.py`.
- 2. Fuente de probabilidades realistas.** Aquí está el reto principal, no el técnico sino el de datos: no hay acceso (según la justificación de la propia tesis) a un feed en tiempo real de incidentes del STC Metro. Opciones viables:
 - Usar reportes históricos públicos (redes sociales oficiales del Metro, prensa, boletines de contingencia) para estimar frecuencia aproximada de incidentes por línea — trabajo de recolección manual/scraping, factible en el tiempo de tesis a escala pequeña (decenas de eventos, no miles).
 - Alternativamente (y es lo recomendable dado el alcance de "prototipo validado con datos sintéticos" que ya declara el anexo), **parametrizar la probabilidad a mano con justificación teórica/documental** (p. ej. "la Línea A reporta X incidentes por trimestre según nota de prensa Y"), dejando explícito que es una aproximación y no una calibración estadística rigurosa. Esto es consistente con el marco metodológico ya aceptado (datos sintéticos, no datos reales en tiempo real).
 - Para lluvia específicamente, sí existe una fuente pública razonable: climatología histórica de la CDMX (SMN/Conagua) para estimar la probabilidad de lluvia fuerte por mes/hora, que es más defendible que inventar una tasa arbitraria.
- 3. Grafo con disponibilidad variable.** El código actual (`grafo_base_metro.gexf` , copiado cada hora) soporta perfectamente eliminar/restaurar aristas por snapshot, ya que cada `G_hora = G_base.copy()` es independiente. No se requiere cambio estructural de librería (sigue siendo NetworkX), solo lógica adicional.
- 4. Impacto en el modelo predictivo.** Añadir eventos estocásticos exige ampliar las features del `RandomForestRegressor` (o migrar a un modelo que maneje mejor variables categóricas/exógenas, ej. Gradient Boosting con variables dummy de evento). El riesgo real es de **datos, no de modelo**: con solo 3,740 filas de un día, meter eventos raros (baja frecuencia por diseño) puede dejar clases extremadamente desbalanceadas. Esto refuerza la necesidad del paso previo (varios días simulados) antes de sumar estocasticidad, para tener suficientes muestras positivas de "hubo evento" en el set de entrenamiento.
- 5. Costo computacional.** Trivial. Simular N días × 19 horas × un sorteo de evento por hora es una operación de milisegundos; el cuello de botella sigue siendo el ruteo de la matriz O-D con `nx.shortest_path` por cada par origen-destino, que ya corre hoy sin problema para un día.

3.3 Riesgos y mitigaciones

- **Riesgo:** sobre-ingeniería del generador de eventos distrayendo del objetivo central (ruteo anticipatorio). *Mitigación:* limitar el generador a 2-3 tipos de evento (lluvia, falla de tramo, cierre de estación) con una distribución simple, documentando explícitamente que el objetivo es probar que el sistema **reacciona correctamente ante disponibilidad variable del grafo**, no producir un simulador meteorológico/operativo exhaustivo.
- **Riesgo:** que la probabilidad "inventada" reste rigor metodológico ante el jurado. *Mitigación:* citar fuente para cada parámetro (aunque sea aproximado) y encuadrarlo explícitamente como supuesto de simulación, tal como ya se hace con los perfiles origen/destino y la función BPR actual — el anexo mismo justifica el uso de simulación por la dificultad de acceso a datos reales en tiempo real.

Veredicto: la integración es factible dentro del alcance de la tesis y no requiere nueva infraestructura ni librerías; el trabajo pendiente es principalmente de diseño de parámetros (tasas de ocurrencia) y de expandir el dataset a múltiples días para que el modelo tenga suficientes ejemplos de eventos disruptivos que aprender.

Mensaje de commit

feat: modelo fijo, 30 semillas y edad del evento

- entrenador_anticipatorio.py: el modelo exportado es fijo (RandomForest + linea/tramo, MODELO_ELEGIDO); la seleccion por RMSE con evento de cada semilla era ruido (30 semillas: 6 ganadores distintos). Nueva feature edad_evento.
- simulador_congestion.py: guarda edad_evento (horas desde hora_inicio, -1 sin evento); la duracion no se expone. Mismos sorteos que antes.
- experimento_semillas.py: 30 semillas; bootstrap con IC de ahorro y perdidas por sistema y probabilidad de que la IA pierda menos. La IA pierde 5.3x menos que el reactivo (IC de la diferencia excluye 0) y captura 47.7% [23.2; 67.6] del ahorro posible; su ventaja en ahorro total sigue sin ser significativa (P = 60%). edad_evento no aporta (ablacion: RMSE identico).
- avances.md: seccion 7; bloques de prompt reescritos como bitacora.